A Smaller Checked Presentation of iPRA

Immutable proof contexts and a scalar neutral premise

Research prepared for Vladimir Reshetnikov

November 11, 2026

Abstract

Under ConciseIPRA's complete-proof-checker metric, three standalone Python modules attain a new coordinate envelope of **256 significant tokens, 278 AST nodes, and 521 normalized bytes**. The corresponding envelope in the inspected current primary checker corpus, including all six incoming archives, is 257 tokens, 279 nodes, and 522 bytes. Different programs attain the coordinates. The improvement is small and concerns the implementation of an inherited axiomatization: a global-scope checker is converted to a loop with immutable contexts, and local-scope checkers use scalar zero as their neutral true premise. Neither change removes a mathematical axiom.

The implemented public calculus is the existing guarded-source presentation. It is equiconsistent and bi-interpretable with full first-order iPRA, with whole numerical domains, actual equality, and identity numerical composites. We state its entire rule inventory and both interpretations, prove the preservation properties of the implementation changes, and retain the inherited mathematical sources. A separate elementary theorem weakens the scalar kernel's equality rule to replacement of one left-side occurrence. Its checker-size benefit is not established. No absolute minimality or new proof-assistant verification is claimed.

1 Result and comparison boundary

The repository was read at commit

`a4e5e96c456303cf231eab2aeeb3291d8889833f`.

The comparison directory is `Logic/PrimitiveRecursiveArithmetic/ConciseIPRA`. The README describes the native scalar calculus, its Lean equivalence with iPRA, and twenty-eight filed research packages. The current tree also contains six incoming ZIP archives. These are part of the comparison, rather than being treated as future work.[1, 2]

The measured improvement is a refinement of `ipra_guarded_source_2026-11-11.zip`, already present in that tree. The source-dispatch reduction, guarded matrices, and the proof that they compose are inherited. The scalar-zero neutral-premise convention also appears in the incoming reversal package; its use here is a composition of existing techniques.[3, 4]

Complete standalone module	Tokens	AST	Bytes	Gzip-9
Incoming token profile	257	285	568	338
New token profile	**256**	283	594	346
Incoming AST profile	261	279	603	345
New AST profile	260	**278**	602	344
Incoming byte profile	264	301	522	328
New byte profile	263	300	**521**	327

These figures count the *entire* module, including JSON input, scope handling, substitution, all proof rules, and acceptance output. The token-oriented change increases its normalized byte length; it

is chosen for the token coordinate. The three coordinate improvements are approximately 0.389%, 0.358%, and 0.192%, respectively. Gzip is informational and is not the optimization criterion.

The source audit screened 151 primary Python files under the filed and incoming `code/` directories, retaining 123 standalone checker candidates. Short generators and support scripts were not counted as checkers. The detailed rows, exclusions, hashes, and runtime information are in `evidence/primary_checker_audit.json`. This is a documented comparison corpus, not a lower bound over all possible presentations.

Input contract. The compact modules assume that the claimed public theorem is supplied in the canonical guarded encoding. They may accept additional sound administrative formulas; they do not validate the entire public grammar. The separately counted `code/checker_strict.py` validates a closed public goal as well. It is a *public-goal* validator, not a claim that every object in the certificate has undergone a separate exact-type check. Its complete size is 1,201 normalized bytes (489 tokens, 560 AST nodes, 571 gzip-9 bytes), versus 1,202 bytes for the identical inherited validator with its old neutral premise. Its metrics and tests are recorded alongside the compact profiles. Comparisons to validators for other public grammars would be comparisons under different contracts.

2 The complete implemented axiomatization

This section fixes the mathematical presentation implemented by the new modules. It is the inherited guarded-source calculus $\mathcal{U}^-$ with its specified public language. All primitive axioms and rules are given here.

2.1 Terms and formulas

There is one numerical sort. Terms are finite expressions

$$t ::= x \mid 0 \mid S(t) \mid I(n, a; u.h).$$

Only u in h is bound. Other variables in h are fixed numerical parameters. Equivalently, each fixed finite earlier-stage body h gives a first-order function symbol I_h. No numerical variable ranges over function descriptions, and there is no uniformly total universal evaluator. Write $1 = S0$ and

$$C(v, a, b) := I(v, a; u.b).$$

Public formulas are equations and paired binders $\mathsf{D}_{x,y}B$. An equation immediately under D must have the exact shape

$$E(x, y) \equiv I(y, 0; u.h) = I(y, 0; u.k). \tag{Guard}$$

The update bodies may depend on x, y, u and passive parameters. If the matrix is compound, its root is another D, and it must itself be public. Every guarded equation has a provable zero instance. The designated contradiction is the public equation $0 = 1$.

Proofs may additionally use the auxiliary formulas

$$A ::= (s = t) \mid \mathsf{Q}_y A \mid \mathsf{D}_{x,y} A,$$

including unguarded atomic matrices. A public sequent has public assumptions and conclusion; its internal proof formulas need not be public. This is an explicit feature of the presentation, and the equivalence theorem below does not assert both round trips for every auxiliary formula.

2.2 All primitive axioms and rules

The only numerical axiom schemas, available in every retained context, are

$$I(0, a; u.h) = a, \tag{I0}$$
$$I(Sn, a; u.h) = h[I(n, a; u.h)/u]. \tag{IS}$$

Substitution is capture-avoiding throughout. There are four additional inference patterns:

Hypothesis.

$$\frac{}{\Gamma \vdash A} \quad (A \in \Gamma). \tag{Hyp}$$

Source-selected elimination. For an auxiliary formula template $F(v)$, define

$$M(s, t; F) = \begin{cases} \mathsf{Q}_v F(v), & s \text{ is literally } 0, \\ s = t, & \text{otherwise.} \end{cases}$$

Then use

$$\frac{\Gamma \vdash M(s, t; F) \qquad \Gamma \vdash F(s)}{\Gamma \vdash F(t)}. \tag{SE}$$

The restriction is syntactic: a nonzero-looking iterator term that denotes zero is an allowed equality source. At literal zero, this rule is anchored closure elimination.

Paired elimination.

$$\frac{\Gamma \vdash \mathsf{D}_{x,y} B(x, y) \qquad \Gamma, \mathsf{Q}_y B(b, y) \vdash A}{\Gamma \vdash A}. \tag{DE}$$

Here b is fresh for Γ, A, and the passive parameters of the matrix. The displayed local assumption is discharged.

Deterministic paired introduction. Put $\eta_B(z) = z$ when B is a literal equation, and $\eta_B(z) = 0$ otherwise. Use

$$\frac{\Gamma, B(t, \eta_B(z)) \vdash B(t, Sz)}{\Gamma \vdash \mathsf{D}_{x,y} B(x, y)}. \tag{DI}$$

The eigenparameter z is fresh for the retained context, the witness t, and passive matrix parameters. Again, the local assumption is discharged.

This inventory is complete. Reflexivity, symmetry, numerical coverage, successor injectivity and separation, equation induction, ordinary connectives, and ordinary quantifiers are derived. In particular, no rule of classical excluded middle or induction on arbitrary formulas is added.

Weakening, assumption cut, and fresh renaming are admissible. Numerical substitution into a restricted-source proof is not used as a primitive metarule: replacing a variable source by literal zero can invalidate a primitive SE step. Later equivalence theorems may establish substitution as a derived operation on the public consequence relation.

3 Why the presentation is exactly iPRA

We give the interpretations and the dependency order, rather than relying only on the assertion that the standard numerical functions are primitive recursive. The detailed inherited proofs are compiled in `foundations.pdf`, with their source modules retained unchanged.

3.1 Auxiliary soundness and public interpretation

Interpret every finite iterator description by its corresponding primitive recursive definition. Preserve numerical equality. Auxiliary formulas have the syntax-sensitive reading

$$\begin{aligned}(s = t)^* &= (s = t),\\ (\mathsf{Q}_y B)^* &= B^*(0) \to \forall y\, B^*(y),\\ (\mathsf{D}_{x,y} B)^* &= \begin{cases} \exists x\, \forall y\, (B(x,y) \to B(x,Sy)), & B \text{ an equation},\\ \exists x\, (B^*(x,0) \to \forall y\, B^*(x,y)), & B \text{ compound}. \end{cases}\end{aligned}$$

Every inference is sound in iPRA. Atomic DI introduces a progressive predicate. Atomic DE uses equation induction to obtain its anchored closure. Compound DI uses the zero-or-successor coverage theorem, and compound DE is existential elimination. SE is equality transport or anchored closure elimination. Formula roots are preserved by numerical substitution, so this translation commutes with that substitution.

The public reading instead interprets every D uniformly by

$$(\mathsf{D}_{x,y} B)^\circ = \exists x\big(B^\circ(x,0) \to \forall y\, B^\circ(x,y)\big). \tag{1}$$

At a guarded equation, $E(x,0)$ is a theorem. Hence iPRA proves

$$\forall y(E(x,y) \to E(x,Sy)) \iff E(x,0) \to \forall y E(x,y).$$

The forward direction uses only equation induction; the reverse uses the true base instance. Structural induction therefore proves $A^* \leftrightarrow A^\circ$ for every public A.

3.2 Completeness without a circular substitution argument

The inherited completeness proof proceeds in the following order.

1. Derive contextual equation induction in the auxiliary calculus. From $E(z) \vdash E(Sz)$, atomic DI with an unused witness gives $\mathsf{D}_{x,y} E(y)$. DE exports $\mathsf{Q}_y E(y)$, and SE applies it to the base $E(0)$ at any desired numerical target. The equation may be unguarded.

2. Work first in the purely equational fragment. Computation and transport with sources other than literal zero, together with the derived equation induction, suffice to reverse $0 = t$ into $t = 0$. The supplied proof uses a represented zero, a singleton test, and an unconditional zero filter. Equation cases come directly from equation induction. The proof keeps passive parameters fixed and does not substitute zero into a derivation whose transport source was a variable.

3. Recover unrestricted numerical equality reasoning and then transport through every public formula template. At a guarded atomic matrix its zero instance is true, so the opened closure supplies the successor instance needed for DI. At a compound matrix, use induction on the template to transport the local base backwards, apply the opened closure, and transport the successor instance forwards.

4. Obtain exactly the consequences of the inherited guarded calculus $\mathcal{G}$ on public contexts and conclusions. In the reverse direction, translate a proof into iPRA using the mixed reading, replace it by the equivalent public reading, and apply the faithful guarded interpretation back. No identity claim for an arbitrary auxiliary formula is used.

The distinction in the last step matters. The administrative syntax serves as a sound proof language; the public syntax carries the bi-interpretation. The supplied mathematical review checks this dependency order explicitly.

3.3 Both translations and their composites

The ordinary scalar theory T_{eq} uses the iterator equations, intuitionistic first-order equality logic, numerical coverage, $0 = 1 \to \bot$, and contextual induction for literal equations. Its inherited arithmetic construction derives predecessor, equality reflection into a Boolean term, quantifier-free induction, pairing and projections, and then a compiler for every fixed primitive-recursive description.[5, 6]

For example, after pairing is available, a primitive recursion

$$f(0, \vec{p}) = g(\vec{p}), \qquad f(Sn, \vec{p}) = h(n, f(n, \vec{p}), \vec{p})$$

is compiled by iterating the state update

$$r \longmapsto \langle S\pi_0 r,\ h(\pi_0 r, \pi_1 r, \vec{p})\rangle$$

from $\langle 0, g(\vec{p})\rangle$, and projecting the second coordinate. The counter and value use one scalar state. The proof of the defining equations and of each round trip is internal equation induction, rather than an external observation about standard numerals.

For the logical translation, let $\mathrm{sel}(v, A, B)$ select A at zero and B at successors, and let $\mathrm{gate}(v, A)$ be true at zero and equivalent to A at successors. Their construction uses the numerical selector C at equational leaves and finite syntactic padding of compound formulas. In the native Eq/Q/D calculus,

$$\begin{aligned}
\bot &:= 0 = 1,\\
A \to B &:= \mathsf{Q}_y\, \mathrm{sel}(y, A, B),\\
A \wedge B &:= \mathsf{Q}_y\, \mathrm{gate}(y, \mathrm{sel}(\mathrm{par}(y), A, B)),\\
\exists x A(x) &:= \mathsf{D}_{x,y}\, \mathrm{gate}(y, A(x)),\\
A \vee B &:= \exists v\ \mathrm{sel}(v, A, B),\\
\forall x A(x) &:= A(0) \wedge \mathsf{Q}_y A(y).
\end{aligned}$$

Here par is a fixed parity iterator. The inherited proofs derive all natural-deduction rules for these macros.

Replace each $\mathsf{Q}_y B$ by $\mathsf{D}_{u,y} B$ with an unused fresh witness u. To make matrices public, for an equation $E \equiv s = t$ put

$$L(E) := \mathsf{D}_{u,v}(C(v, 0, s) = C(v, 0, t)), \qquad P(A) := \begin{cases} L(A), & A \text{ an equation},\\ A, & A \text{ compound}. \end{cases}$$

The padding translation is

$$T(s = t) = s = t, \qquad T(\mathsf{D}_{x,y} B) = \mathsf{D}_{x,y} P(TB).$$

Both padding composites are internally interderivable with the original formula; the guarded atomic case uses its true zero instance.

Let Φ be the iPRA-to-public translation obtained by composing the scalar compiler, logical macros, dummy-witness expansion, and padding. Let Ψ be the public ordinary reading followed by expansion

of each scalar iterator into its primitive-recursive definition. Then

$$\begin{aligned}&\mathsf{iPRA} \vdash \varphi \leftrightarrow \Psi\Phi(\varphi),\\ &\quad A \vdash_{\mathcal{U}^-} \Phi\Psi(A) \quad \text{and} \quad \Phi\Psi(A) \vdash_{\mathcal{U}^-} A.\end{aligned}$$

The second line holds for every public A. The interpretations use the whole numerical domains and actual equality, and the graph of each numerical composite is $x = y$. Consequently

$$\mathcal{U}^- \vdash 0 = 1 \quad \Longleftrightarrow \quad \mathsf{iPRA} \vdash \bot.$$

These are the same interpretations for every new checker: the implementation changes do not change the public consequence relation.

4 The implementation reductions and their proofs

4.1 Certificate conventions

Each module reads a JSON array from standard input. Its last item is the goal; preceding instruction packets are consumed from right to left. A packet is $[m, s, u, t]$. The checker either closes a retained hypothesis, uses a selected elimination premise, supplies a computation equation, or introduces a paired formula. An unused prefix is inert. Acceptance means normal process exit with exactly `ACCEPT` and a newline. Exceptions, nonzero exit, and resource failure are not acceptance. Assertions are not logical gates.

The inherited arity codec uses unary arrays for D formulas, binary arrays for equations, and ternary arrays for administrative Q closures. Numerical zero is scalar 0, successor is unary, and the iterator is ternary. The opening traversal increments its depth by $5 - \text{arity}$; numerical roots use depth 3. The encoding's protected and dummy coordinate conventions are specified in the retained arity source and certificate documentation. Extra raw shapes have an administrative interpretation; they are not silently new public formulas.

There are two protocols. Local profiles carry the fresh-variable floor as an argument of each call. The token profile uses a shared global floor. They do not in general accept the same serialized certificate. The archive labels and tests the two sets separately.

4.2 Tail-call elimination with immutable contexts

The incoming token checker ends a non-hypothesis step by recursively checking the continuation with an extended context. Replace that tail call by a loop and use a tuple for the retained context:

```
while i(a) not in g:
    # Check the instruction and its major premise.
    ...
    k -= 1
    g += (i(h, (m or len(a) == 2) and k),)
    a = i(a, [k])
```

The complete measured source is `code/checker_tokens.py`; this illustration is not the counted program.

Theorem 4.1 (Continuation preservation)**.** *Under ideal unbounded finite-certificate execution, replacing the incoming token checker's final recursive call by this loop and replacing contexts by tuples preserves its successful proof executions, packet consumption, and global fresh-name state.*

Proof. Relate an old list context and a new tuple context when they have the same entries in the same order. Membership tests agree. A major-premise call receives related contexts and the same packet stream and global floor; by induction on its finite successful call tree it returns after consuming the same packets with the same floor. The next decrement is therefore the same in both executions.

The old final call receives the opened goal and the old context concatenated with one opened hypothesis. The loop next checks precisely those objects with the same global floor. Tuple concatenation creates a new context value and rebinds only the local name. It does not change the context retained by the caller of a major-premise proof. The pure opening operations are evaluated in the opposite order in the two continuation spellings, but neither changes the packet stream, context, or global floor. On successful finite executions both finish and give identical objects.

When a hypothesis closes, both versions return `None`; the old tail frames then only return `None` as well. This gives both simulation directions on finite successful executions. □

Fixed CPython stack limits need not give identical resource failures: the loop can check some longer valid continuations. The theorem compares proof recognition under the same ideal execution convention as the baseline.

Why a tuple is necessary. Using a list with `g += [hypothesis]` would mutate a context shared with the caller and permit a local assumption to escape. The included regression certificate

```
[[[],0,[],[0]],[[[0,[0]]],0,0,0],[0,[0]]]
```

claims the false public equation $0 = 1$. The correct tuple checker rejects it. The deliberately constructed mutable-list mutant accepts it. This is a test of a logical scope property, not merely a comparison of syntax counts.

4.3 Scalar neutral premise

The local profiles initially retain a true padding formula. Replace its spelling `[]` by scalar `0`, and use `0` in the neutral major field of local certificates:

```
def c(a, g=[0], k=0):
    ...
```

The tuple-context AST profile analogously uses `g=(0,)`.

Proposition 4.2 (Neutral-premise preservation)**.** *The scalar-neutral local profiles remain sound for their inherited raw administrative reading and complete for the same public guarded consequences.*

Proof. Both neutral spellings mean truth at formula sort. For a neutral major, the elimination match fails in either spelling, the recursive premise check closes immediately, its tripled export is false as a Python condition, and the fresh-assumption selector takes the same branch. Therefore a canonical proof certificate is translated by replacing neutral-major fields only; other fields and the public goal remain unchanged.

In canonical derivation certificates every meaningful formula goal has nonempty root arity, which opening preserves. It cannot be confused with the new scalar neutral assumption. This proves completeness by translating the existing certificate construction. For arbitrary raw certificates, the only added initial resource is still a true formula, and all actual rule checks are unchanged. The inherited raw soundness argument therefore applies with the new true resource. □

This proposition is not a claim that the two modules accept exactly the same raw JSON language. Their neutral premise spelling differs. The translation acts on packet fields, not on every empty array occurring inside a formula or term.

A restricted-substitution detail. The arity checker's computation-supply proof admits fresh numerical parameters and subsequently specializes them to zero. In this restricted calculus, make that specialization *after* interpreting the continuation in ordinary iPRA, where numerical substitution is valid. Then cut the known computation instance. Do not apply an unrestricted substitution theorem directly to a restricted-source derivation. The inherited guarded-source proof uses exactly this order.

5 An auxiliary mathematical weakening of equality

The following result concerns the unrestricted-source native scalar kernel $\mathcal{N}$ formalized in the repository. It is independent of the measured implementation improvement and does not replace the implemented SE rule.

Definition 5.1 (Left-occurrence replacement)**.** Retain all rules of $\mathcal{N}$ except equational transport, and replace that rule by

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash C[s] = r}{\Gamma \vdash C[t] = r}, \tag{LOR}$$

where $C[-]$ is a capture-avoiding numerical term context with exactly one hole; the identity context is allowed. The right endpoint r is fixed.

Theorem 5.2 (One occurrence suffices)**.** *For every native context and formula, the LOR calculus and $\mathcal{N}$ derive exactly the same sequents. Their term and formula translations are identities.*

Proof. Write $j_a = I(0, a; u.0)$. Computation gives $j_a = a$. Use this equation as both premises of LOR with the identity context to obtain $a = a$. From $s = t$ and $s = s$, LOR gives $t = s$.

Given an old transport step with template $A(v) = B(v)$ and premises $s = t$, $A(s) = B(s)$, enumerate the distinguished free occurrences of v in A, and replace them one at a time by LOR. This gives $A(t) = B(s)$. Apply the just-derived symmetry, replace the distinguished occurrences in B one at a time, and symmetrize again. The result is $A(t) = B(t)$. Before replacement, rename bound iterator variables fresh from s, t and the passive parameters. Occurrences are tracked in the template, not found by replacing every subtree that happens to equal s.

Conversely, LOR is an instance of the old rule with equation template $C[v] = r$. Replacing each old transport node by the finite construction above translates derivations in either direction, in every retained context. □

The repository's native/iPRA bi-interpretation transfers immediately to this alternative kernel. The primitive equality interface is more restricted, but it has not been shown to yield a smaller complete checker: the old checker shares formula opening between transport and closure elimination. A new occurrence-position traversal has a real cost. This is elementary equality reasoning, and the earlier counter-congruence package already studies related restrictions in another numerical language.

6 A tempting reduction introduces full induction

The root test distinguishing equation progress from compound cases is a logical boundary. An attempted implementation reduction uses the generated hypothesis's arity instead of both the major mode and the matrix root. Administrative closures have arity three, so that reduction also permits progress at Q-rooted matrices.

Proposition 6.1 (Closure progress yields unrestricted induction)**.** *With the ordinary native reading of* Q*, permitting progress introduction at arbitrary* Q*-rooted matrices yields induction for every native formula.*

Proof. For an arbitrary $P(n)$ define

$$G(n) := \mathsf{Q}_z \,\mathrm{gate}(z, P(n)).$$

The gate is true at zero and equivalent to $P(n)$ at each successor, so $G(n) \dashv\vdash P(n)$. A proof of $P(n) \vdash P(Sn)$ therefore gives $G(n) \vdash G(Sn)$. The proposed progress introduction, with a dummy witness, followed by DE and QE supplies induction for G. Its base and conclusion translate respectively to $P(0)$ and $P(t)$. Since P was arbitrary, the result is full formula induction. □

The strengthening reaches public Eq/D formulas through the existing translation. Restricting only public goal syntax therefore does not repair it. One can instead distinguish object formulas and closure resources by a genuine type discipline, or prove a normalization theorem allowing only hypothesis checks at closure goals. Those costs must be included in the checker; merely giving the same raw object different meanings in different uses fails to justify closure elimination.

7 Verification, reproduction, and further questions

The archive contains the three compact modules, an expanded standalone checker, a public-goal validator, frozen comparison modules, the original global and local certificates and their translated forms, and a standard- library reproduction script. The script measures complete modules and runs positive and negative controls normally and with Python optimization. It also executes the context-leak regression against the correct checker and a temporary mutant.

The final reproduction run completed **210 candidate/control comparisons** (420 Python processes) and four additional scope-control executions: **424 processes in total, with zero discrepancies from their prescribed outcomes**. The correct checker rejects the scope attack in both modes; the deliberate mutable-list mutant accepts it in both.

The checks include computations, reflexivity, a used equation-induction hypothesis, compound case introduction, zero-source transport, malformed packets, false equations, scope leakage, and future-witness capture. The receipts list every tested process and verdict. These finite executions support the implementation changes; they do not replace the universal proof arguments or constitute a Lean/Rocq verification.

The existing Lean native/iPRA bridge was read at the pinned commit. It was not rebuilt here. The guarded-source bridge is inherited written mathematics, independently reviewed in this run. The review notes state their scope explicitly; they are not external human peer review.

Reproduce. From the extracted archive root, run

```
python3 reproduce.py
pdflatex -interaction=nonstopmode -halt-on-error report.tex
pdflatex -interaction=nonstopmode -halt-on-error report.tex
pdflatex -interaction=nonstopmode -halt-on-error foundations.tex
pdflatex -interaction=nonstopmode -halt-on-error foundations.tex
pdflatex -interaction=nonstopmode -halt-on-error foundations.tex
```

The checker and reproduction commands require Python's standard library. The PDF commands require the ordinary LaTeX packages named in the sources. The archive records CPython and zlib versions because normalized source and compression measurements can depend on them.

Further research questions

1. Can one complete module beat all three current coordinate minima simultaneously, without changing the canonical-goal contract? The present result uses separate programs.
2. Can public syntax validation share the kernel's opening traversal while still checking its binding conventions completely? This is a more substantial engineering target than another one-byte permissive reduction.
3. Can the guarded-source dependency theorem and both public composites be added to the existing Lean formalization, with the restricted substitution boundary explicit?
4. Can closure resources be given an intrinsic representation that permits a uniform continuation rule at lower total cost? A proof must include all eliminations, not only the introduction rule.
5. Does left-occurrence transport help a readable or formally verified checker even when it does not help the compact arity implementation?
6. Is there a different arithmetic foundation that removes an operation and also reduces the complete checker? This investigation has not established such a mathematical improvement.

References

[1] Vladimir Reshetnikov and ProveIt contributors, *Concise presentations of intuitionistic primitive recursive arithmetic*, pinned README, commit `a4e5e96c456303cf231eab2aeeb3291d8889833f`. https://github.com/VladimirReshetnikov/ProveIt/blob/a4e5e96c456303cf231eab2aeeb3291d8889833f/Logic/PrimitiveRecursiveArithmetic/ConciseIPRA/README.md.

[2] ProveIt, *ConciseIPRA incoming proposals*, same commit. All six ZIP archives were inspected; exact member provenance and primary checker measurements are supplied with this report. https://github.com/VladimirReshetnikov/ProveIt/tree/a4e5e96c456303cf231eab2aeeb3291d8889833f/Logic/PrimitiveRecursiveArithmetic/ConciseIPRA/docs/incoming.

[3] ProveIt research package, *A smaller guarded presentation of iPRA*, member `report.tex` of `ipra_guarded_source_2026-11-11.zip` in the preceding directory. Source of the unchanged implemented calculus, mixed auxiliary reading, equational dependency theorem, public consequence theorem, and three comparison modules.

[4] ProveIt research package, *A smaller checked presentation of iPRA*, `ipra_reversal_core_2026-11-11.zip`, same incoming directory. Source of the previously used scalar-zero neutral-premise convention.

[5] ProveIt, *certified_zero/reference*, *guarded_core*, and *source_dispatch*, same pinned commit. Attributed frozen mathematical sources are in `reference/foundations/`; the supplementary `foundations.pdf` compiles the required mathematical chain. https://github.com/VladimirReshetnikov/ProveIt/tree/a4e5e96c456303cf231eab2aeeb3291d8889833f/Logic/PrimitiveRecursiveArithmetic/ConciseIPRA/Research.

[6] ProveIt contributors, *ConciseIPRA Lean development*, especially `NativeBiInterpretation.lean`, `BiInterpretation.lean`, and `NativeZeroSource.lean`, same commit. https://github.com/VladimirReshetnikov/ProveIt/tree/a4e5e96c456303cf231eab2aeeb3291d8889833f/Logic/PrimitiveRecursiveArithmetic/ConciseIPRA/Lean/ConciseIPRA.